



DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

MODELLING AND CONTROL OF MANIPULATORS

First Assignment

Equivalent representations of orientation matrices

Author:

Beaini Charbel
Topulli Joel

Student ID:

S8027823
S8663382

Professors:

Enrico Simetti
Giorgio Cannata

Tutors:

Luca Tarasi
Simone Borelli

November 11, 2025

Contents

1 Assignment description	3
1.1 Exercise 1 - Angle-Axis to Rotation Matrix	3
1.2 Exercise 2 - Rotation Matrix to Angle-Axis	3
1.3 Exercise 3 - Euler Angles to Rotation Matrix	4
1.4 Exercise 4 - Rotation Matrix to Euler Angles	4
1.5 Exercise 5 - Rotation Matrix to Angle-Axis using Eigenvectors	4
1.6 Exercise 6 - Frame tree	5
2 Exercise 1	6
2.1 Q1.1	6
2.2 Q1.2	6
2.3 Q1.3	6
2.4 Q1.4	6
3 Exercise 2	7
3.1 Q2.1	7
3.2 Q2.2	8
3.3 Q2.3	8
3.4 Q2.4	8
3.5 Q2.5	8
3.6 Q2.6	9
4 Exercise 3	9
4.1 Q3.1	9
4.2 Q3.2	10
4.3 Q3.3	10
4.4 Q3.4	10
4.5 Q3.5	10
5 Exercise 4	11
6 Exercise 5	12
7 Exercise 6	13

Mathematical expression	Definition	MATLAB expression
$\langle w \rangle$	World Coordinate Frame	w
${}^a_b R$	Rotation matrix of frame $\langle b \rangle$ with respect to frame $\langle a \rangle$	aRb
${}^a_b T$	Transformation matrix of frame $\langle b \rangle$ with respect to frame $\langle a \rangle$	aTb

Table 1: Nomenclature Table

1 Assignment description

The first assignment of Modelling and Control of Manipulators focuses on the geometric fundamentals and algorithmic tools underlying any robotics application. The concepts of transformation matrix, orientation matrix and the equivalent representations of orientation matrices (Equivalent angle-axis representation and Euler Angles) will be reviewed.

The first assignment is **mandatory** and consists of 5 different exercises. You are asked to:

- Download the .zip file called MCM-LAB1 from the Aulaweb page of this course.
- Implement the code to solve the exercises on MATLAB by filling the predefined files called "main.m", "AngleAxisToRot.m", "RotToAngleAxis.m", "YPRToRot.m", "RotToYPR.m" and "IsRotationMatrix.m".
- Write a report motivating the answers for each exercise, following the predefined format on this document.
- The usage of built-in MATLAB functions is strictly forbidden except for basic mathematical operations such as *det*, *eig*.

1.1 Exercise 1 - Angle-Axis to Rotation Matrix

A particularly interesting minimal representation of 3D rotation matrices is the so-called angle-axis representation, where a rotation is represented by the axis of rotation $\mathbf{h}$ and the angle θ . Any rotation matrix can be represented by its equivalent angle-axis representation by applying the Rodrigues Formula.

Q1.1 Given an angle-axis pair $(\mathbf{h}, \theta)$, implement on MATLAB the Rodrigues formula, computing the equivalent rotation matrix, **WITHOUT** using built-in matlab functions. The function signature will be

$$\text{function } R = \text{AngleAxisToRot}(h, \theta)$$

Then test it for the following cases and briefly comment the results obtained:

- **Q1.2** $\mathbf{h} = [1, 0, 0]^T$ and $\theta = 90^\circ$
- **Q1.3** $\mathbf{h} = [0, 0, 1]^T$ and $\theta = \pi/3$
- **Q1.4** $\rho = [-\pi/3, -\pi/6, \pi/3]$;

1.2 Exercise 2 - Rotation Matrix to Angle-Axis

Given a rotation matrix R , the problem of finding the corresponding angle-axis representation $(\mathbf{h}, \theta)$ is called the Inverse Equivalent Angle-Axis Problem.

Q2.1 Given a rotation matrix R , implement on MATLAB the Equivalent Angle-Axis equations **WITHOUT** using built-in matlab functions. The function signature will be

$$\text{function } [h, \theta] = \text{RotToAngleAxis}(R)$$

You **MUST** check that the input is a valid rotation matrix by filling in and utilizing the function *IsRotationMatrix(R)*.

Hint: utilize a suitable tolerance (e.g. 10^{-3}) to check the properties of the matrices.

Test it for the following cases and briefly comment the results obtained:

- **Q2.2** $R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$
- **Q2.3** $R = \begin{pmatrix} 0.5 & -\sqrt{3}/2 & 0 \\ \sqrt{3}/2 & 0.5 & 0 \\ 0 & 0 & 1 \end{pmatrix}$
- **Q2.4** $R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$
- **Q2.5** $R = \begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

• **Q2.6**
$$R = \begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

1.3 Exercise 3 - Euler Angles to Rotation Matrix

Any orientation matrix can be expressed in terms of three elementary rotations in sequence. Consider the Yaw Pitch Roll (YPR) representation, where the sequence of the rotation axes is Z-Y-X.

Q3.1 Given a triplet of YPR angles (ψ, θ, ϕ) , compute the equivalent rotation matrix representation **WITHOUT** using built-in matlab functions. The function signature will be

$$\text{function } R = \text{YPRToRot}(\text{psi}, \text{theta}, \text{phi})$$

Then test it for the following cases and briefly comment the results obtained:

- **Q3.2** $\psi = \theta = 0, \phi = \pi/2$
- **Q3.3** $\phi = \theta = 0, \psi = 60^\circ$
- **Q3.4** $\psi = \pi/3, \theta = \pi/2, \phi = \pi/4$
- **Q3.5** $\psi = 0, \theta = \pi/2, \phi = -\pi/12$

1.4 Exercise 4 - Rotation Matrix to Euler Angles

Given a rotation matrix R , it is possible to compute an equivalent triplet of YPR angles (ψ, θ, ϕ) , provided that the configuration is not singular (that is, $\cos \theta \neq 0$).

Q4.1 Given a rotation matrix R , implement in MATLAB the equivalent YPR angles, **WITHOUT** using built-in matlab functions. The function signature will be

$$\text{function } [\text{psi}, \text{theta}, \text{phi}] = \text{RotToYPR}(R)$$

You **MUST** check that the input is a valid rotation matrix by filling in and utilizing the function *IsRotationMatrix*(R). **Hint:** utilize a suitable tolerance (e.g. 10^{-3}) to check the properties of the matrices.

Test it for the following cases and briefly comment the results obtained:

- **Q4.2**
$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$$
- **Q4.3**
$$R = \begin{pmatrix} \frac{1}{2} & -\frac{\sqrt{3}}{2} & 0 \\ \frac{\sqrt{3}}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 1 \end{pmatrix}$$
- **Q4.4**
$$R = \begin{pmatrix} 0 & -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 0.5 & \frac{\sqrt{2}\sqrt{3}}{4} & \frac{\sqrt{2}\sqrt{3}}{4} \\ -\frac{\sqrt{3}}{2} & \frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} \end{pmatrix}$$

1.5 Exercise 5 - Rotation Matrix to Angle-Axis using Eigenvectors

Consider a rotation matrix R and the corresponding angle-axis representation $(\mathbf{h}, \theta)$. Then, $R\mathbf{h} = \mathbf{h}$, which can be verified by computing $R\mathbf{h}$ with the Rodrigues Formula and observing that the terms containing $\mathbf{h} \times \mathbf{h}$ vanish. Therefore, $\mathbf{h}$ is an eigenvector of R corresponding to the eigenvalue +1.

Given the following rotation matrices, compute the corresponding angle-axis representation with two different methods. In the first method, compute $(\mathbf{h}, \theta)$ using the function *RotToAngleAxis* implemented before. In the second one, compute $\mathbf{h}$ as the eigenvector of R corresponding to eigenvalue +1, and then use the function *RotToAngleAxis* to compute θ . Compare and briefly discuss the results.

• **Q5.1**
$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$$

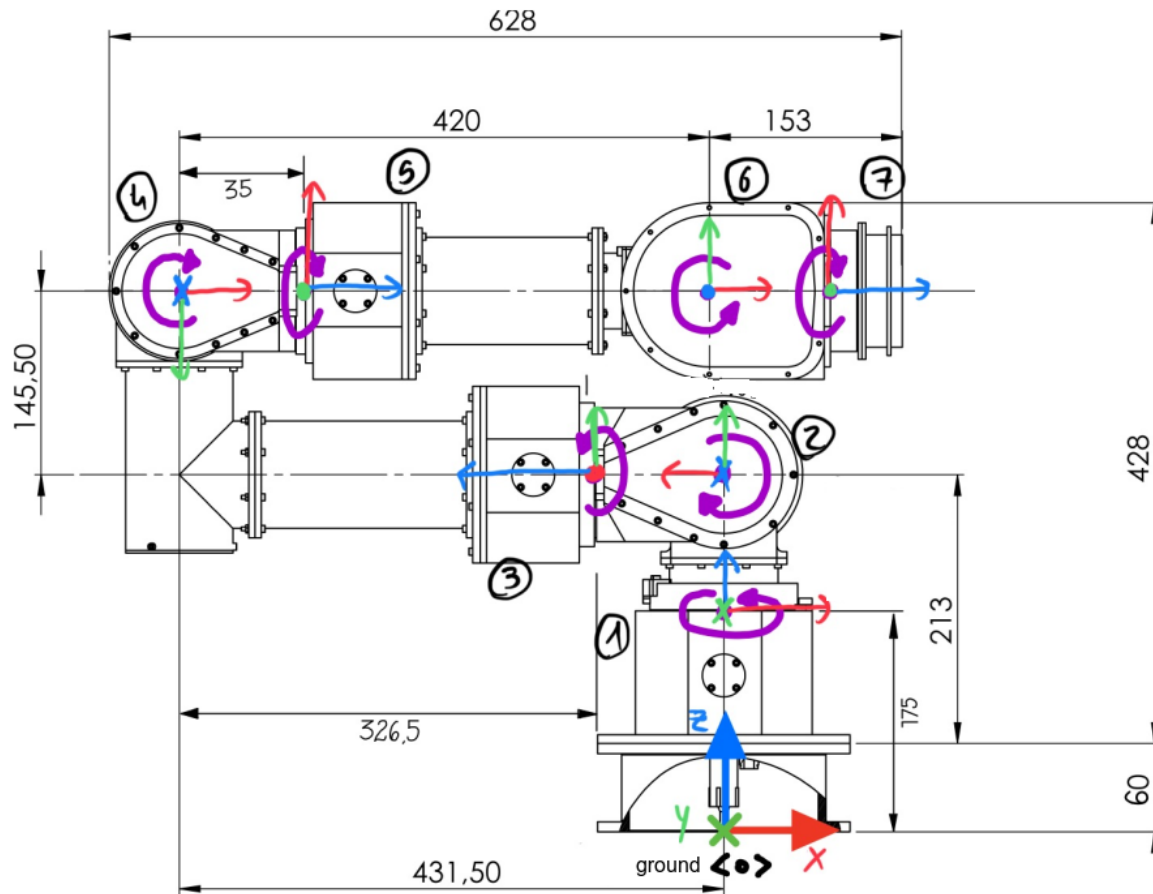


Figure 1: Exercise 6 frames. The unit of measurement is millimeters.

• Q5.2
$$R = \frac{1}{9} \begin{pmatrix} 4 & -4 & -7 \\ 8 & 1 & 4 \\ -1 & -8 & 4 \end{pmatrix}$$

1.6 Exercise 6 - Frame tree

Figure 1 shows the frame tree for the 7 joints of the Franka robot. With reference to the figure, use the geometric definition of the transformation matrix to compute by hand the following matrices.

- Q6.1 0_1T
- Q6.2 1_2T
- Q6.3 2_3T
- Q6.4 3_4T
- Q6.5 4_5T
- Q6.6 5_6T
- Q6.7 6_6T
- Q6.8 7_6T

You **MUST** compute the matrices **WITHOUT** using mathematical software.

2 Exercise 1

In three-dimensional rotational kinematics, a rotation can be represented compactly by an axis-angle pair $(\mathbf{h}, \theta)$. The direction of $\mathbf{h}$ specifies the axis of rotation and the scalar θ defines the rotation magnitude. To convert this representation into a 3×3 rotation matrix, we apply the Rodrigues' rotation formula, shown in (1). This provides a closed-form expression for the rotation matrix using only the unit axis, its associated skew-symmetric matrix, and trigonometric functions of θ .

2.1 Q1.1

For our function `AngleAxisToRot( $\mathbf{h}, \theta$ )`, we need to implement the Rodrigues' formula (1) to turn the given angle-axis parameters into a rotation matrix. First we ensure that the axis of rotation $\mathbf{h}$ is non-zero so we can compute its norm. After unitaryizing the axis vector we build the corresponding skew-symmetric matrix $[\mathbf{h}]_{\times}$. Then we create the symmetric and anti-symmetric components and apply:

$$R = I + [\mathbf{h}]_{\times} \sin \theta + [\mathbf{h}]_{\times}^2 (1 - \cos \theta) \quad (1)$$

We also treat edge cases. If $\mathbf{h} = 0$ and $\theta = 0$, the identity matrix is returned. If $\mathbf{h} = 0$ and $\theta \neq 0$, the input is invalid.

2.2 Q1.2

For $\mathbf{h} = [1, 0, 0]^T$ and $\theta = 90^\circ$, the rotation matrix that we obtain is:

$$R = \begin{pmatrix} 1.0000 & 0 & 0 \\ 0 & 0.0000 & -1.0000 \\ 0 & 1.0000 & 0.0000 \end{pmatrix}$$

This corresponds to a rotation of 90° about the x -axis. Vectors along the y -axis are rotated toward the z -axis, while the x -axis remains unchanged.

2.3 Q1.3

For $\mathbf{h} = [0, 0, 1]^T$ and $\theta = \pi/3$ (60°), the rotation matrix is:

$$R = \begin{pmatrix} 0.5000 & -0.8660 & 0 \\ 0.8660 & 0.5000 & 0 \\ 0 & 0 & 1.0000 \end{pmatrix}$$

This represents a rotation of 60° about the z -axis. The vectors on the xy -plane rotate counterclockwise, while the z -axis remains unchanged.

2.4 Q1.4

In this case, we are provided with the rotation vector $\boldsymbol{\rho}$ rather than the angle-axis parameters. The vector is:

$$\boldsymbol{\rho} = [-\pi/3; -\pi/6; \pi/3]$$

First, we compute the rotation angle θ as the norm of $\boldsymbol{\rho}$. We then check its validity. If $\theta = 0$, the rotation axis $\mathbf{h}$ is arbitrary, therefore we decide to express it as the following unit vector:

$$\mathbf{h} = [1; 0; 0]$$

Otherwise, if $\theta \neq 0$, the unit rotation axis is computed as:

$$\mathbf{h} = \frac{\boldsymbol{\rho}}{\theta}$$

Using these angle–axis parameters, we can construct the corresponding rotation matrix. In our case $\theta \neq 0$, and along with h it produces the following matrix:

$$R = \begin{pmatrix} 0.4444 & -0.4444 & -0.7778 \\ 0.8889 & 0.1111 & 0.4444 \\ -0.1111 & -0.8889 & 0.4444 \end{pmatrix}$$

3 Exercise 2

3.1 Q2.1

`RotToAngleAxis(R)` takes a 3×3 rotation matrix and returns angle–axis parameters. First it verifies that R is a valid element of $SO(3)$ by calling `IsRotationMatrix(R)`. The check uses the principles of:

a) Orthonormality:

$$R^T R = I \quad (2)$$

b) Determinant:

$$\det(R) = +1 \quad (3)$$

If the matrix fails either condition, the function returns: $\theta = \text{NaN}$ and $h = \text{NaN}$. This prevents undefined values from propagating. But if the matrix is valid, we compute θ :

$$\theta = \arccos\left(\frac{\text{tr}(R) - 1}{2}\right) \quad (4)$$

Edge cases:

1) $\theta = 0$: The rotation is the identity. Any unit vector is a valid axis, so our implementation returns:

$$h = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

2) $\theta = \pi$: This is the most complex case due to the fact that we need to calculate the magnitudes and the signs separately. We start by computing the squared components:

$$h_x^2 = \frac{R_{11} + 1}{2}, \quad h_y^2 = \frac{R_{22} + 1}{2}, \quad h_z^2 = \frac{R_{33} + 1}{2} \quad (5)$$

Subsequently we quantify the magnitudes and then afterwards we determine the signs. Since a rotation axis has two possible orientations ($\pm h$) we need to work out the "correct" one. Hence by combining the signs of the magnitudes with the signs of the off-diagonals, (in our case we chose $\text{sign}(R(1, 2))$) we produce a direction. Consequently the magnitudes are modified accordingly so they abide by the mathematical restrictions that are imposed by the aforementioned direction.

3) $0 < \theta < \pi$: This is a general case which allows us to use the standard formula:

$$h = \frac{1}{2 \sin \theta} \text{vex}(R - R^T) \quad (6)$$

Where we know that:

$$\text{vex}(M) = \begin{bmatrix} M_{32} \\ M_{13} \\ M_{21} \end{bmatrix} \quad (7)$$

And the result will be a unit axis vector.

3.2 Q2.2

We are given the rotation matrix R as shown below:

$$R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$$

The result of our calculation is:

$$h = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \theta = 1.5708$$

This corresponds to a 90° rotation about the x -axis. The case is regular ($0 < \theta < \pi$), so the result is obtained through the standard formula (6).

3.3 Q2.3

We are given the rotation matrix R as shown below:

$$R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$$

The function returns

$$h = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad \theta = 1.0472$$

Due to the values of θ and h we can determine that the transformation corresponds to a 60° rotation about the z -axis. Therefore, we can say that the case is regular ($0 < \theta < \pi$).

3.4 Q2.4

The rotation matrix that has been provided is the 3×3 identity matrix shown below:

$$R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

In our results, we see the message:

'h is not uniquely defined therefore it can be represented by any unit vector'

This tells us that we are in the case where $\theta = 0$ and the axis of rotation h can be chosen arbitrarily. Our interpretation is further confirmed by the output:

$$h = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \theta = 0$$

3.5 Q2.5

The given rotation matrix is:

$$R = \begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

And our function returns:

$$\mathbf{h} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \quad \theta = 3.1416$$

Corresponding to a rotation of π radians about the z -axis which allows us to come to the conclusion that the x - and y -axes are flipped, while the z -axis remains unchanged. We have to mention that we are in the special case where $\theta = \pi$; hence the magnitudes and the signs of the elements that make up h were calculated separately so they uphold the restrictions imposed by the chosen direction.

3.6 Q2.6

We are provided the following matrix:

$$R = \begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

When we execute our function, `IsRotationMatrix` is also called inside of it. And in this instance it indicates that this matrix is not a valid rotation matrix because the determinant condition is not satisfied. Therefore, this is a special case, and we raise an error. To avoid leaving the angle-axis parameters undefined, we represent them as:

$$\mathbf{h} = \begin{pmatrix} \text{NaN} \\ \text{NaN} \\ \text{NaN} \end{pmatrix}, \quad \theta = \text{NaN}$$

4 Exercise 3

Yaw-pitch-roll (YPR) angles provide a common parametric description of 3D orientation in robotics, graphics, and aerospace. Each angle represents a rotation around one of the principal axes, and the complete orientation is obtained by applying these rotations sequentially. Converting YPR angles to a rotation matrix requires building the individual rotations around the z -axis (yaw), y -axis (pitch), and x -axis (roll), then multiplying them in the specified intrinsic order. Because rotation matrices do not commute, changing this order produces a different orientation.

4.1 Q3.1

The function `YPRToRot( $\psi, \theta, \phi$ )` computes a 3×3 rotation matrix which corresponds to a sequence of yaw-pitch-roll (YPR) rotations; that can be expressed as:

Yaw (ψ) rotation around z -axis:

$$R_z(\psi) = \begin{pmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (8)$$

Pitch (θ) rotation around y -axis:

$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{pmatrix} \quad (9)$$

Roll (ϕ) rotation around x -axis:

$$R_x(\phi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{pmatrix} \quad (10)$$

The final rotation matrix is obtained by the multiplication of these matrices in the following order:

$$R = R_z(\psi) R_y(\theta) R_x(\phi) \quad (11)$$

The order is intrinsic because using a different sequence of multiplications would result in a different rotation matrix.

4.2 Q3.2

The Euler angles are given as:

$$\psi = \theta = 0, \quad \phi = \pi/2$$

The resulting rotation matrix is:

$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$$

This shows that only a roll rotation about the x -axis has been applied. The x -axis remains unchanged, while the y -axis is rotated into the negative z -direction and the z -axis into the positive y -direction.

4.3 Q3.3

Given the Euler angles:

$$\phi = 0, \quad \theta = 0, \quad \psi = 60^\circ$$

The resulting rotation matrix obtained from the YPR sequence is:

$$R = \begin{pmatrix} 0.5000 & -0.8660 & 0 \\ 0.8660 & 0.5000 & 0 \\ 0 & 0 & 1.0000 \end{pmatrix}$$

4.4 Q3.4

The provided Euler angles are:

$$\psi = \frac{\pi}{3}, \quad \theta = \frac{\pi}{2}, \quad \phi = \frac{\pi}{4}$$

Therefore the resulting rotation matrix from the YPR sequence is:

$$R = \begin{pmatrix} 0.0000 & -0.2588 & 0.9659 \\ 0.0000 & 0.9659 & 0.2588 \\ -1.0000 & 0.0000 & 0.0000 \end{pmatrix}$$

This rotation corresponds to a pitch of $\theta = \pi/2$, which aligns the y -axis with the z -axis. The yaw and roll rotations then rotate vectors in the plane orthogonal to the x -axis. The x -axis remains unchanged, while the y - and z -axes are rotated accordingly.

4.5 Q3.5

Given the Euler angles:

$$\psi = 0, \quad \theta = \frac{\pi}{2}, \quad \phi = -\frac{\pi}{12}$$

The resulting rotation matrix from the YPR sequence is:

$$R = \begin{pmatrix} 0.0000 & -0.2588 & 0.9659 \\ 0.0000 & 0.9659 & 0.2588 \\ -1.0000 & -0.0000 & 0.0000 \end{pmatrix}$$

Here, we observe the same rotation matrix as in the previous case. This occurs because Euler angles are not unique: when the pitch angle θ is near $\pi/2$, the yaw and roll rotations become dependent due to the alignment of axes. Adjusting ψ and ϕ by equal and opposite amounts produces the same rotation, which is a known phenomenon called *gimbal lock*.

5 Exercise 4

Given a rotation matrix $R \in \mathbb{R}^{3 \times 3}$, the objective is to determine the equivalent set of yaw-pitch-roll (YPR) angles (ψ, θ, ϕ) , assuming that the configuration is not singular, i.e.

$$\cos \theta \neq 0. \quad (12)$$

The adopted convention is defined as follows: ψ denotes the rotation about the z -axis (yaw), θ the rotation about the y -axis (pitch), and ϕ the rotation about the x -axis (roll).

Q4.1

The MATLAB script file *RotToYPR* is conceived to map a proper rotation matrix into its corresponding YPR parameterization without resorting to built-in MATLAB functions for Euler angle extraction.

The first step performed by the function is the validation of the input matrix through `IsRotationMatrix`. This check ensures that R is orthogonal

$$RR^T = I, \quad (13)$$

and has unit determinant (within the order of 10^{-3}).

Once the matrix has been validated, the extraction of the angles is done following the standard YPR decomposition. The pitch angle θ is obtained first from the element R_{31} , exploiting the relation

$$\theta = \text{atan2}(-R_{31}, \sqrt{R_{11}^2 + R_{21}^2}). \quad (14)$$

After θ has been determined, two situations are distinguished:

- **Regular case** ($|\theta| \not\approx \frac{\pi}{2}$): when the configuration is far from a gimbal-lock condition, the remaining two angles can be uniquely calculated as

$$\psi = \text{atan2}(R_{21}, R_{11}), \quad \phi = \text{atan2}(R_{32}, R_{33}). \quad (15)$$

- **Gimbal-lock case** ($|\theta| \approx \frac{\pi}{2}$): when the pitch approaches $\pm 90^\circ$, the rotation about one axis becomes unobservable and the decomposition is no longer unique. In this situation the function explicitly detects the singularity (again using a small tolerance) and assigns a conventional value to one of the angles (the roll ϕ is set to zero). The yaw ψ is then recovered directly from the first row of R ; the sign in the $\text{atan2}(\cdot, \cdot)$ expression differs for $+90^\circ$ and -90° in order to compensate for the opposite pitch and keep the overall orientation consistent. This guarantees that the function still returns a consistent triplet, even if the geometric configuration is ill-conditioned.

Q4.2

For the provided rotation matrix:

$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix} \quad (16)$$

The function returned:

$$\psi = 0, \quad \theta = 0, \quad \phi = 1.570796 \text{ rad}. \quad (17)$$

This matrix represents a rotation of 90° about the x -axis. Since a pure roll should produce zero yaw and zero pitch, the obtained YPR triplet is consistent with the geometry of the transformation.

Q4.3

For:

$$R = \begin{pmatrix} \frac{1}{2} & -\frac{\sqrt{3}}{2} & 0 \\ \frac{\sqrt{3}}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (18)$$

The function returned:

$$\psi = 1.047198 \text{ rad}, \quad \theta = 0, \quad \phi = 0. \quad (19)$$

This matrix is a planar rotation about the z -axis by 60° (i.e. $\frac{\pi}{3}$ rad). In the YPR convention, a pure rotation about z must appear entirely in the yaw angle, with pitch and roll equal to zero. The numerical output therefore matches the expected physical rotation.

Q4.4

For:

$$R = \begin{pmatrix} 0 & -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 0.5 & \frac{\sqrt{2}\sqrt{3}}{4} & \frac{\sqrt{2}\sqrt{3}}{4} \\ -\frac{\sqrt{3}}{2} & \frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} \end{pmatrix} \quad (20)$$

The our result is:

$$\psi = 1.570796 \text{ rad}, \quad \theta = 1.047198 \text{ rad}, \quad \phi = 0.7853982 \text{ rad}. \quad (21)$$

This is a non-special 3D rotation, and the recovered angles correspond approximately to:

$$\psi \approx \frac{\pi}{2}, \quad \theta \approx \frac{\pi}{3}, \quad \phi \approx \frac{\pi}{4}, \quad (22)$$

that is, a 90° yaw, a 60° pitch, and a 45° roll. Since the matrix does not satisfy the singularity condition, the algorithm can use the regular extraction formulas, and the three angles are uniquely identified.

6 Exercise 5

The aim of this exercise is to calculate the angle-axis representation $(\mathbf{h}, \theta)$ associated with a given rotation matrix $R \in \mathbb{R}^{3 \times 3}$ using two conceptually different procedures and to compare the results. As stated in the given text, for any proper rotation matrix R and its corresponding rotation axis $\mathbf{h}$, it holds

$$R\mathbf{h} = \mathbf{h}, \quad (23)$$

which means that $\mathbf{h}$ is an eigenvector of R associated with the eigenvalue $+1$. This follows directly from the Rodrigues formula, since the term involving $\mathbf{h} \times \mathbf{h}$ is zero. Consequently, the rotation axis can be obtained either (i) analytically from the skew-symmetric part of R , or (ii) as the eigenvector corresponding to the unit eigenvalue.

In what follows, the function *RotToAngleAxis*, previously introduced and explained in Section 3, is used as the reference implementation. An additional function *RotToAngleAxisEigen* uses the same output angle θ (taken from *RotToAngleAxis* to avoid computing the same quantity twice), but obtains the axis $\mathbf{h}$ as the normalized eigenvector of R associated with eigenvalue $+1$. This provides a convenient way to verify the consistency of the two approaches.

Q5.1

For

$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$$

the two methods gave the following results.

Using *RotToAngleAxis*:

$$\mathbf{h} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \theta = 1.5708 \text{ rad}.$$

Using *RotToAngleAxisEigen*:

$$\mathbf{h}_{\text{eigen}} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \theta_{\text{eigen}} = 1.5708 \text{ rad}.$$

The matrix represents a rotation of 90° about the x -axis. Both procedures identify the x -axis as the invariant direction and assign the same rotation angle. Since this case is neither numerically ill-conditioned nor close to the special case $\theta = \pi$, the two methods have exactly the same output.

Q5.2

For

$$R = \frac{1}{9} \begin{pmatrix} 4 & -4 & -7 \\ 8 & 1 & 4 \\ -1 & -8 & 4 \end{pmatrix}$$

the two methods produced the results below.

Using *RotToAngleAxis*:

$$\mathbf{h} = \begin{pmatrix} -0.6667 \\ -0.3333 \\ 0.6667 \end{pmatrix}, \quad \theta = 1.5708 \text{ rad.}$$

Using *RotToAngleAxisEigen*:

$$\mathbf{h}_{\text{eigen}} = \begin{pmatrix} 0.6667 \\ 0.3333 \\ -0.6667 \end{pmatrix}, \quad \theta_{\text{eigen}} = 1.5708 \text{ rad.}$$

Here the two axes differ only by sign, while the angle is identical. This behaviour is expected, because the angle-axis representation is not unique:

$$(\mathbf{h}, \theta) \quad \text{and} \quad (-\mathbf{h}, -\theta) \quad (24)$$

In this implementation, *RotToAngleAxis* determines $\mathbf{h}$ from the skew-symmetric part of R as depicted in Section 3, whereas the eigenvector-based approach directly returns an eigenvector associated with the eigenvalue 1, whose direction is defined only up to a sign. After normalization, both vectors have unit length and, together with the common angle $\theta = \frac{\pi}{2}$, represent the same physical rotation. As discussed in class, the eigenvector-based method can recover the correct components of the axis but cannot fix its direction uniquely; both $\mathbf{h}$ and $-\mathbf{h}$ are valid eigenvectors. For this reason, it is generally preferable to use the *RotToAngleAxis* formulation to obtain a deterministic angle-axis pair from a rotation matrix.

7 Exercise 6

In this exercise, the frame hierarchy associated with the seven joints of the Franka robot is explored. As shown in the handwritten derivations, the rotation part of each transform is obtained first by composing elementary rotations (one axis at a time), and only afterward is the translation part computed. By applying the geometric definition of homogeneous transforms, the following transformation matrices were determined. The standard definitions of the elementary rotation matrices about the coordinate axes were adopted to build all the rotations in Exercise 6. The right-hand rule is applied to determine the sign of the rotation angle about each axis.

$$R_x(\phi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{pmatrix}, \quad (25)$$

$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{pmatrix}, \quad (26)$$

$$R_z(\psi) = \begin{pmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{pmatrix}. \quad (27)$$

Equations (25)-(27) provide the general form of the rotations about the x -, y -, and z -axes, respectively, and are the basis for all subsequent rotation computations.

A homogeneous transform has the form

$${}^a_bT = \begin{pmatrix} {}^a_bR & {}^a_r_{a/b} \\ 0 & 1 \end{pmatrix}, \quad (28)$$

where ${}^a_bR \in \mathbb{R}^{3 \times 3}$ is the rotation matrix and ${}^a_r_{a/b} \in \mathbb{R}^{3 \times 1}$ is the translation vector.

Q6.1

0_1T is obtained as

$${}^0_1T = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 175 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

There is no rotation between frame 0 and frame 1, therefore the rotational part is the identity matrix. For the translational part, frame 1 is located 175 mm from frame 0 along the z_0 direction, which is reflected in the fourth column.

Q6.2

Starting by composing the elementary rotations around each axis, the following rotation matrices are

$$R_z(-\pi) = \begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad R_x\left(\frac{\pi}{2}\right) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}.$$

Multiplying them in that order, the result below is obtained

$${}^1_2R = R_z(-\pi) R_x\left(\frac{\pi}{2}\right) = \begin{pmatrix} -1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}.$$

The corresponding homogeneous transformation is

$${}^1_2T = \begin{pmatrix} -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 98 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

In this case the rotation first flips the x -axis and then permutes the y - z directions as shown above. After the rotation, frame 2 is translated by 98 mm along its new z -axis.

Q6.3

Starting from the elementary rotation around the y -axis

$$R_y\left(\frac{\pi}{2}\right) = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{pmatrix}.$$

Since only one rotation is needed in this step, the rotation matrix is directly

$${}^2_3R = R_y\left(\frac{\pi}{2}\right).$$

The corresponding homogeneous transformation is

$${}^2_3T = \begin{pmatrix} 0 & 0 & 1 & 105 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Here the rotation maps the original x_2 axis onto z_3 and the original z_2 onto $-x_3$. After the rotation, frame 3 is translated by 105 mm along the x -axis of frame 2.

Q6.4

Starting by composing the elementary rotations, the following rotation matrices are

$$R_z(-\pi) = \begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad R_y\left(\frac{\pi}{2}\right) = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{pmatrix}.$$

Multiplying them in that order yields the following result

$${}^3_4R = R_z(-\pi) R_y\left(\frac{\pi}{2}\right) = \begin{pmatrix} 0 & 0 & -1 \\ 0 & -1 & 0 \\ -1 & 0 & 0 \end{pmatrix}.$$

The corresponding homogeneous transformation is

$${}^3_4T = \begin{pmatrix} 0 & 0 & -1 & 0 \\ 0 & -1 & 0 & 145.5 \\ -1 & 0 & 0 & 326.5 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

In this case the rotation first flips the x - y plane (via $R_z(-\pi)$) and then rotates about y so that the new z_4 aligns with $-x_3$. After the rotation, frame 4 is translated by 145.5 mm along y_3 and by 326.5 mm along z_3 , as shown in the fourth column.

Q6.5

Starting by composing the elementary rotations, the following rotation matrices are

$$R_y\left(\frac{\pi}{2}\right) = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{pmatrix}, \quad R_z\left(-\frac{\pi}{2}\right) = \begin{pmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Multiplying them in that order yields the following result

$${}^4_5R = R_y\left(\frac{\pi}{2}\right) R_z\left(-\frac{\pi}{2}\right) = \begin{pmatrix} 0 & 0 & 1 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \end{pmatrix}.$$

The corresponding homogeneous transformation is

$${}^4_5T = \begin{pmatrix} 0 & 0 & 1 & 35 \\ -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Here the rotation realigns the x_5 axis with the previous z_4 , while flipping the remaining axes as required by the task. After the rotation, frame 5 is translated by 35 mm along its new x -axis.

Q6.6

This transformation is also obtained by composing two elementary rotations:

$$R_x\left(-\frac{\pi}{2}\right) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{pmatrix}, \quad R_z\left(-\frac{\pi}{2}\right) = \begin{pmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Multiplying them in that order, the result below is obtained

$${}^5_6R = R_x\left(-\frac{\pi}{2}\right) R_z\left(-\frac{\pi}{2}\right) = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}.$$

The corresponding homogeneous transformation is

$${}^5_6T = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 385 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

In this step the rotation permutes the axes so that the new z_6 points along the previous x_5 . After the rotation, frame 6 is translated by 385 mm along this new z direction.

Q6.7

In this case there is no relative motion between the two frames, so the rotation is the identity

$${}^6_6R = I_3,$$

and the corresponding homogeneous transformation is

$${}^6_6T = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

This simply expresses frame 6 with respect to itself.

Q6.8

The corresponding homogeneous transformation is

$${}^7_6T = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & -153 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Here it can be noticed that, the rotation permutes the axes in the same way as in **Q6.6**, and the translation shifts the frame by -153 mm along the new z -axis, bringing it back along the link.

Check & Visualization

Finally, the *PlotRefFrames* MATLAB script was implemented in order to verify the hand-computed transforms. All helper functions, such as *plotFrame* (which draws the triad of a reference frame and labels its axes) and *homog* (which constructs a 4×4 homogeneous matrix from a rotation R and a translation r), were explicitly implemented and do not rely on external toolboxes or built-in functions. The resulting 3D scene depicted in Figure 2 matches the frame tree shown in Figure 1, so the matrices obtained by hand are correct.

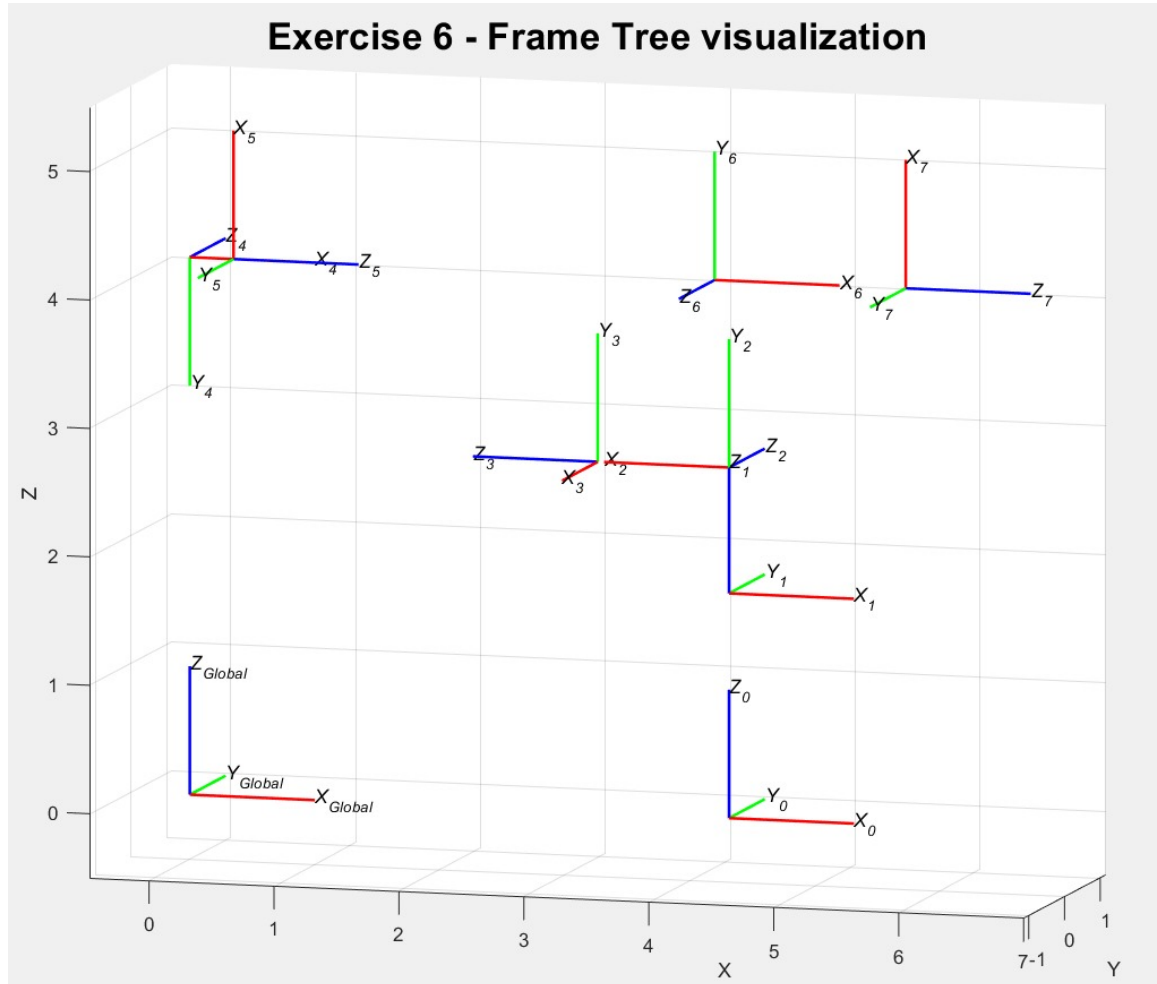


Figure 2: Exercise 6 frames computed on MATLAB using *PlotRefFrames*. The unit of measurement is meters.